

# Simulazione della Diffusione del Calore 2D

Si implementi in C/C++ una simulazione della **diffusione del calore su una griglia 2D** usando il metodo delle differenze finite esplicite. La simulazione modella come il calore si propaga nel tempo su una piastra rettangolare con sorgenti di calore fisse (es. un oggetto caldo al centro) e condizioni al contorno assegnate.

## Background

L'equazione del calore 2D è:

$$\partial T / \partial t = \alpha * (\partial^2 T / \partial x^2 + \partial^2 T / \partial y^2)$$

Discretizzata con differenze finite esplicite diventa:

$$T[i][j](t+1) = T[i][j](t) + \alpha * dt / dx^2 * (T[i+1][j] + T[i-1][j] + T[i][j+1] + T[i][j-1] - 4 * T[i][j](t))$$

## Input

- Dimensioni della griglia  $NX \times NY$  (es.  $512 \times 512$ ,  $1024 \times 1024$ )
- Coefficiente di diffusione  $\alpha$  (default: 0.25, garantisce stabilità numerica)
- Numero di iterazioni temporali
- Condizioni iniziali: bordi a temperatura 0, sorgente calda al centro ( $T=100$ )

## Architettura richiesta

- **Decomposizione del dominio (MPI):** la griglia viene suddivisa in strisce orizzontali, una per processo MPI. Ogni processo gestisce  $NX/size$  righe più due righe fantasma (halo).
- **Halo exchange (MPI\_Sendrecv):** ad ogni iterazione ogni processo scambia le righe di bordo con i vicini prima di applicare lo stencil.
- **Calcolo locale (OpenMP parallel for):** il loop sulle righe del proprio blocco viene parallelizzato con OpenMP.
- **Convergenza (MPI\_Allreduce):** ogni processo calcola l'errore massimo locale  $\max |T_{new} - T_{old}|$ ; tramite `MPI_Allreduce(MPI_MAX)` si verifica se la soluzione è andata a convergenza globalmente.

## Output richiesto

- Griglia di temperatura  $T[i][j]$  salvata in CSV ogni  $N$  iterazioni, visualizzabile con Python/matplotlib
- Errore di convergenza in funzione delle iterazioni

- Numero di iterazioni necessarie alla convergenza al variare di  $\alpha$

### **Metriche da riportare**

1. Throughput: MUPS (Million Updates Per Second) al variare di processi  $\times$  thread
2. Strong scaling ed efficienza parallela
3. Weak scaling: aumentare la griglia proporzionalmente ai processi
4. Impatto della dimensione della griglia sul tempo di halo exchange vs tempo di calcolo

### **Deliverable**

- Codice C/C++ commentato con Makefile
- Script Python per la visualizzazione della mappa di calore e dei grafici di scaling
- Relazione o presentazione con: descrizione dello stencil, architettura parallela, grafici di scaling, analisi della convergenza